UNIBALSAS

Centro Universitário

Curso de Análise e Desenvolvimento de Sistemas

Disciplina: Algoritmos e Programação Avançada

TED 03

Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)

Relatório técnico

Vinicius Muniz Lopes dos Santos

R.A: 26.1.14496

Balsas – MA

2026

1. Identificação

Campo	Informação
Instituição	UniBalsas Centro Universitário
Curso	Análise e Desenvolvimento de Sistemas (ADS)
Disciplina	Algoritmos e Programação Avançada
Título da atividade	TED 03 – Gerenciador de Persistência de Dados em Arquivos Estruturados (CSV e JSON)
Integrante	Vinicius Muniz Lopes dos Santos (R.A: 26.1.14496)
Linguagem	Python 3 (biblioteca padrão)
Ambiente de desenvolvimento	Visual Studio Code
Repositório	https://github.com/vmunizx/TED-03

2. Introdução e objetivos

Quase todo programa precisa guardar informações para usá-las depois. Quando os dados não ficam em um banco, o caminho mais comum é gravá-los em arquivos de texto estruturados, e dois formatos aparecem o tempo todo: o CSV, muito usado em planilhas, e o JSON, padrão na troca de dados entre sistemas. Trabalhar com esses arquivos parece simples, mas traz riscos: o arquivo pode não existir, pode estar com a codificação errada, pode estar mal formatado ou pode ser sobrescrito por engano, e com isso os dados antigos se perdem.

Este trabalho apresenta um módulo em Python que gerencia dados persistidos em arquivos CSV e JSON. O módulo lê os dois formatos, converte um para o outro, grava os resultados com segurança e cria uma cópia de proteção antes de alterar qualquer arquivo existente. Como exemplo de dados, usamos um cadastro simples de clientes (identificador, nome, e-mail, cidade e valor).

Objetivos

- Ler arquivos CSV e JSON e convertê-los de um formato para o outro, preservando os registros e os valores dos campos.
- Usar o gerenciador de contexto `with open(...)` em todas as aberturas de arquivo, com a codificação UTF-8 definida.
- Tratar as exceções de entrada e saída mais comuns, exibindo mensagens claras e sem encerrar o programa de forma inesperada.

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

- Criar um backup automático antes de qualquer escrita ou atualização, e cancelar a operação caso o backup não seja possível.
- Organizar o código em módulos com responsabilidades separadas e validar o funcionamento com testes manuais e automatizados.

3. Descrição da solução

3.1 Organização do código

Separamos o projeto em módulos pequenos, cada um com uma única responsabilidade. Assim, a leitura, a escrita, a conversão, o backup e o tratamento de erros podem ser entendidos e testados de forma independente. A Tabela 1 resume a função de cada arquivo.

Arquivo	Responsabilidade
main.py	Menu interativo e execução direta das opções (`python main.py 1`).
config.py	Caminhos das pastas (dados, erros e backups) e codificações usadas na leitura e na escrita.
leitor.py	Funções `ler_csv` e `ler_json`.
escritor.py	Funções `escrever_csv` e `escrever_json`. Chamam o backup antes de gravar.
conversor.py	Conversão CSV para JSON e JSON para CSV, com ajuste de tipos numéricos.
backup.py	Cópia de segurança com data e hora e a exceção `BackupError`.
erros.py	Tradução das exceções em mensagens claras e função `executar`, que evita o encerramento do programa.
testes/test_persistencia.py	13 testes automatizados com `unittest`.
dados/	Arquivos de exemplo (`clientes.csv`, `clientes.json`) e, em `dados/erros/`, arquivos criados para provocar erros.

Tabela 1 – Módulos do projeto. Fonte: elaborada pelos autores (2026).

3.2 Funcionalidades implementadas

- Leitura de CSV com cabeçalho e de JSON no formato de lista de objetos.
- Conversão CSV para JSON e JSON para CSV sem perda de registros.
- Escrita em UTF-8, com acentos preservados em ambos os formatos.
- Backup automático com data e hora no nome do arquivo, dentro da pasta `backups/`.

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

- Tratamento de arquivo inexistente, falta de permissão, erro de codificação, JSON malformado, CSV inválido e demais erros de E/S.
- Menu com opções que demonstram cada cenário, inclusive os de erro, e listagem dos backups criados.

3.3 Como os dados são manipulados e persistidos

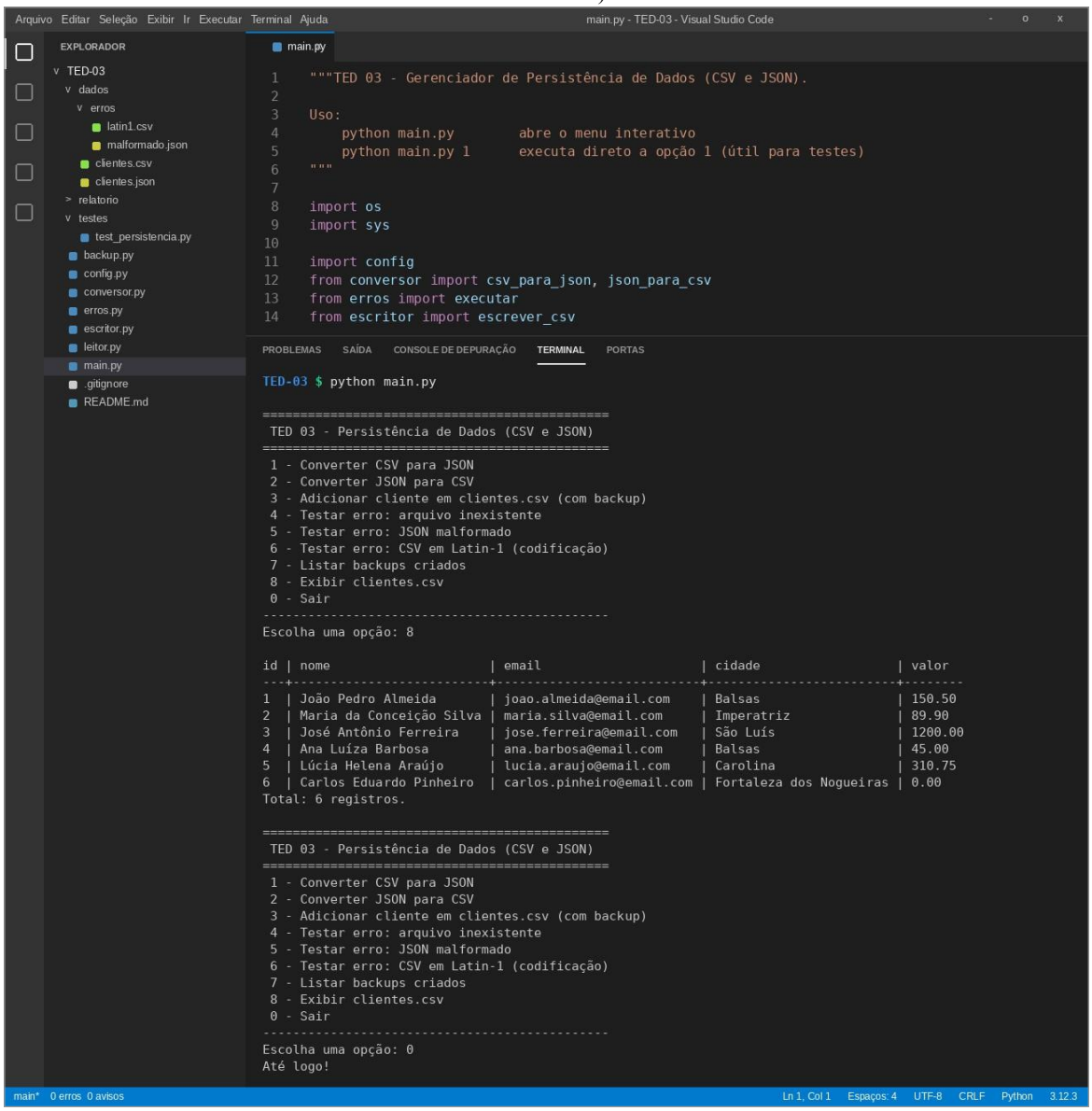
No CSV, a primeira linha traz o cabeçalho (id, nome, e-mail, cidade, valor) e cada linha seguinte é um cliente. Na leitura, cada linha vira um dicionário Python (`csv.DictReader`). No JSON, os dados ficam em uma lista de objetos, um para cada cliente, lida com `json.load`. Como o CSV guarda tudo como texto, a conversão para JSON transforma o campo `id` em número inteiro e o campo `valor` em número decimal. Quando um valor não puder ser convertido, ele permanece como texto, de modo que nenhuma informação é descartada.

Na volta, de JSON para CSV, os números decimais são gravados com duas casas. Essa decisão evita uma perda sutil: sem ela, o valor `150.50` voltaria como `150.5`. O teste de "ida e volta" (seção 5.5) confirma que o CSV original e o CSV regenerado ficam idênticos.

Toda gravação segue a mesma sequência: (1) verificar se o arquivo de destino já existe; (2) se existir, criar o backup; (3) somente se o backup der certo, abrir o arquivo com `with open` e gravar os dados. A Figura 1 mostra o menu do programa em execução.

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

Figura 1 – Menu principal do programa executado no terminal do Visual Studio Code, com a opção 8 (exibir clientes.csv)



Fonte: elaborada pelos autores (2026).

4. Implementação

Esta seção apresenta os trechos mais importantes do código-fonte. Os trechos foram copiados diretamente dos arquivos do repositório.

4.1 Gerenciador de contexto with open(...)

Todas as aberturas de arquivo do projeto usam `with open(...)` com a codificação informada. O bloco `with` fecha o arquivo automaticamente ao terminar, e isso vale também quando acontece uma exceção no meio da operação, o que evita arquivos abertos e recursos presos. A leitura usa

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

`utf-8-sig`, que é UTF-8 e ainda aceita o marcador BOM que o Excel costuma incluir; a escrita usa UTF-8 puro.

leitor.py

```
def ler_csv(caminho):     """Lê um CSV com cabeçalho. Retorna (lista_de_campos,
lista_de_registros)."""     with open(caminho, "r", encoding=ENCODING_LEITURA,
newline="") as arquivo:
        leitor = csv.DictReader(arquivo, strict=True)
if not leitor.fieldnames:
            raise ValueError(f"O arquivo '{caminho}' está vazio ou sem
cabeçalho.")         campos = list(leitor.fieldnames)         registros =
[dict(linha) for linha in leitor]     return campos, registros

def ler_json(caminho):     """Lê um JSON que contenha uma lista de objetos (um
objeto por registro)."""     with open(caminho, "r",
encoding=ENCODING_LEITURA) as arquivo:         dados = json.load(arquivo)

    if not isinstance(dados, list) or not all(isinstance(item, dict) for item in dados):
        raise ValueError("O JSON precisa conter uma lista de objetos
(registros).")     return dados
```

4.2 Leitura, escrita e conversão entre CSV e JSON

A leitura de CSV devolve os campos do cabeçalho e a lista de registros; a de JSON confere se o conteúdo é uma lista de objetos. A conversão de CSV para JSON lê os registros, ajusta os tipos e entrega o resultado ao escritor. A conversão no sentido contrário reúne todos os campos que aparecem nos objetos, na ordem em que surgem, e monta as linhas do CSV.

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

conversor.py

```
def _converter_tipos(registro):
    novo = dict(registro)
    for campo in CAMPOS_INTEIROS:
        try:
            novo[campo] = int(novo[campo])
        except (KeyError, TypeError, ValueError):
            pass
    for campo in CAMPOS_DECIMAIS:
        try:
            novo[campo] = float(str(novo[campo]).replace(",", "."))
        except (KeyError, TypeError, ValueError):
            pass
    return novo

def csv_para_json(origem, destino, pasta_backup=None, avisar=True):
    """Lê o CSV de `origem` e grava o JSON em `destino`. Retorna a quantidade de registros."""
    _, registros = ler_csv(origem)
    convertidos = [_converter_tipos(r) for r in registros]
    escrever_json(destino, convertidos, pasta_backup, avisar)
    return len(convertidos)

def json_para_csv(origem, destino, pasta_backup=None, avisar=True):
    """Lê o JSON de `origem` e grava o CSV em `destino`. Retorna a quantidade de registros."""
    registros = ler_json(origem)

    campos = []  # união dos campos, na ordem em que aparecem
    for registro in registros:
        for campo in registro:
            if campo not in campos:
                campos.append(campo)

    linhas = [{c: _para_texto(r.get(c)) for c in campos} for r in registros]
    escrever_csv(destino, campos, linhas, pasta_backup, avisar)
    return len(linhas)
```

As funções de escrita gravam em UTF-8. No JSON, o parâmetro `ensure_ascii=False` mantém as letras acentuadas legíveis no arquivo (por exemplo, “Conceição”, e não a sequência de escape correspondente). No CSV, o uso de `newline=""` evita linhas em branco extras no Windows. Repare que a primeira linha de cada função é a chamada ao backup.

escritor.py

```
def escrever_json(caminho, registros, pasta_backup=None, avisar=True):
"""Grava a lista de registros em JSON (UTF-8, acentos preservados)."""     backup
= criar_backup(caminho, pasta_backup, avisar)  # 1°: proteger o original
_garantir_pasta(caminho)     with open(caminho, "w", encoding=ENCODING_ESCRITA)
as arquivo:         json.dump(registros, arquivo, ensure_ascii=False, indent=2)
return backup

def escrever_csv(caminho, campos, registros, pasta_backup=None, avisar=True):
"""Grava os registros em CSV com cabeçalho (UTF-8)."""     backup =
criar_backup(caminho, pasta_backup, avisar)  # 1°: proteger o original
_garantir_pasta(caminho)     with open(caminho, "w", encoding=ENCODING_ESCRITA,
newline="") as arquivo:
        escritor = csv.DictWriter(arquivo,
fieldnames=campos)          escritor.writeheader()
escritor.writerows(registros)     return backup
```

4.3 Tratamento de exceções relacionadas às operações com arquivos

Concentramos o tratamento de erros no módulo `erros.py`. A função `executar` roda uma operação dentro de um `try/except`; se algo falhar, ela mostra a mensagem apropriada e

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

devolve o resultado da operação como "falhou", sem derrubar o programa. A função `mensagem_erro` converte cada exceção em um texto que explica o problema. A ordem das verificações importa: as exceções mais específicas, como `UnicodeDecodeError` e `JSONDecodeError`, precisam ser testadas antes de `ValueError`, porque herdam dela.

erros.py

```
def mensagem_erro(erro, caminho=None):     """Transforma uma
exceção em uma mensagem amigável."""     onde = f"
'{config.relativo(caminho)}'" if caminho else ""

    if isinstance(erro, BackupError):
        return f"{erro} A operação foi cancelada e nada foi gravado."
if isinstance(erro, FileNotFoundError):
        return f"Arquivo{onde} não encontrado. Confira o nome e a pasta."
if isinstance(erro, PermissionError):
        return f"Sem permissão para acessar ou modificar o arquivo{onde}."
if isinstance(erro, IsADirectoryError):
        return f"O caminho{onde} é uma pasta, não um arquivo."
if isinstance(erro, UnicodeDecodeError):
        return (f"Erro de codificação ao ler o arquivo{onde}: ele não está em UTF-8
"                f"(byte inválido na posição {erro.start}). "
"Salve o arquivo novamente como UTF-8.")     if isinstance(erro,
json.JSONDecodeError):
        return (f"JSON malformado no arquivo{onde}: {erro.msg}
"                f"(linha {erro.lineno}, coluna
{erro.colno}).")     if isinstance(erro, csv.Error):
        return f"CSV inválido no arquivo{onde}: {erro}"
if isinstance(erro, OSError):
        return f"Erro de entrada/saída no arquivo{onde}: {erro.strerror or
erro}"     if isinstance(erro, ValueError):         return f"Dados
inválidos{onde}: {erro}"     return f"Erro inesperado: {erro}"

def executar(operacao, *argumentos, caminho=None, **opcoes):     """Executa
`operacao`. Retorna (True, resultado) ou (False, None) se houver erro."""     try:
        return True, operacao(*argumentos, **opcoes)     except
(OSError, ValueError, csv.Error, BackupError) as erro:
        print(f"[ERRO] {mensagem_erro(erro, caminho)}")
return False, None
```

Exceção	Quando ocorre	Mensagem apresentada (resumo)
FileNotFoundError	O arquivo pedido não existe.	Arquivo ... não encontrado. Confira o nome e a pasta.
PermissionError	Sem permissão para ler ou gravar.	Sem permissão para acessar ou modificar o arquivo ...
UnicodeDecodeError	O arquivo não está em UTF-8 (por exemplo, Latin-1).	Erro de codificação ... byte inválido na posição N.
json.JSONDecodeError	O JSON está malformado.	JSON malformado ... (linha L, coluna C).
csv.Error	O CSV está inválido.	CSV inválido no arquivo ...
ValueError	Arquivo vazio ou JSON que não é lista de objetos.	Dados inválidos ...

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

BackupError	Não foi possível criar o backup.	... A operação foi cancelada e nada foi gravado.
OSError (outras)	Demais falhas de entrada e saída.	Erro de entrada/saída no arquivo ...

Tabela 2 – Exceções tratadas e mensagens exibidas ao usuário. Fonte: elaborada pelos autores (2026).

4.4 Mecanismo de backup automático

A função `criar_backup` é chamada pelo escritor antes de qualquer gravação. Se o arquivo de destino ainda não existe, não há o que proteger e o programa apenas informa isso. Se existe, ele é copiado com `shutil.copy2` (que preserva também as datas do arquivo) para a pasta `backups/`, com o nome no formato `nome_AAAAMMDD_HHMMSS.extensão`. Um contador evita que dois backups feitos no mesmo segundo se sobrescrevam. O usuário é avisado quando o backup é criado.

O ponto mais importante é o comportamento em caso de falha: se a cópia não puder ser feita, a função levanta `BackupError`. Como a exceção é lançada antes da abertura do arquivo de destino, a gravação nem começa e o arquivo original permanece intacto. Esse comportamento é conferido por um teste automatizado (seção 5.5).

backup.py

```
class BackupError(Exception):
    """Falha ao criar o backup. Quando isso acontece, nada deve ser gravado."""

def criar_backup(caminho, pasta_backup=None, avisar=True):
    """Copia `caminho` para a pasta de backups, com data e hora no nome.

    Retorna o caminho do backup criado, ou None quando o arquivo ainda não
    existe (nesse caso não há nada para proteger).
    Levanta BackupError se a cópia não puder ser feita.
    """
    if not os.path.exists(caminho):
        if avisar:
            print(f"[INFO] '{config.relativo(caminho)}' ainda não existe; "
                  "backup não é necessário.")
        return None

    pasta = pasta_backup or config.PASTA_BACKUP
    nome, extensao = os.path.splitext(os.path.basename(caminho))
    carimbo = datetime.now().strftime("%Y%m%d_%H%M%S")

    try:
        os.makedirs(pasta, exist_ok=True)
        destino = os.path.join(pasta, f"{nome}_{carimbo}{extensao}")
        contador = 1
        # Evita sobrescrever um backup feito no mesmo segundo.
        while os.path.exists(destino):
            destino = os.path.join(pasta, f"{nome}_{carimbo}_{contador}{extensao}")
            contador += 1
        shutil.copy2(caminho, destino)
    except OSError as erro:
        raise BackupError(
            f"Não foi possível criar o backup de '{config.relativo(caminho)}': {erro}"
        ) from erro

    if avisar:
        print(f"[BACKUP] Cópia de segurança criada em '{config.relativo(destino)}'.")
    return destino
```


Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

5. Testes e evidências de execução

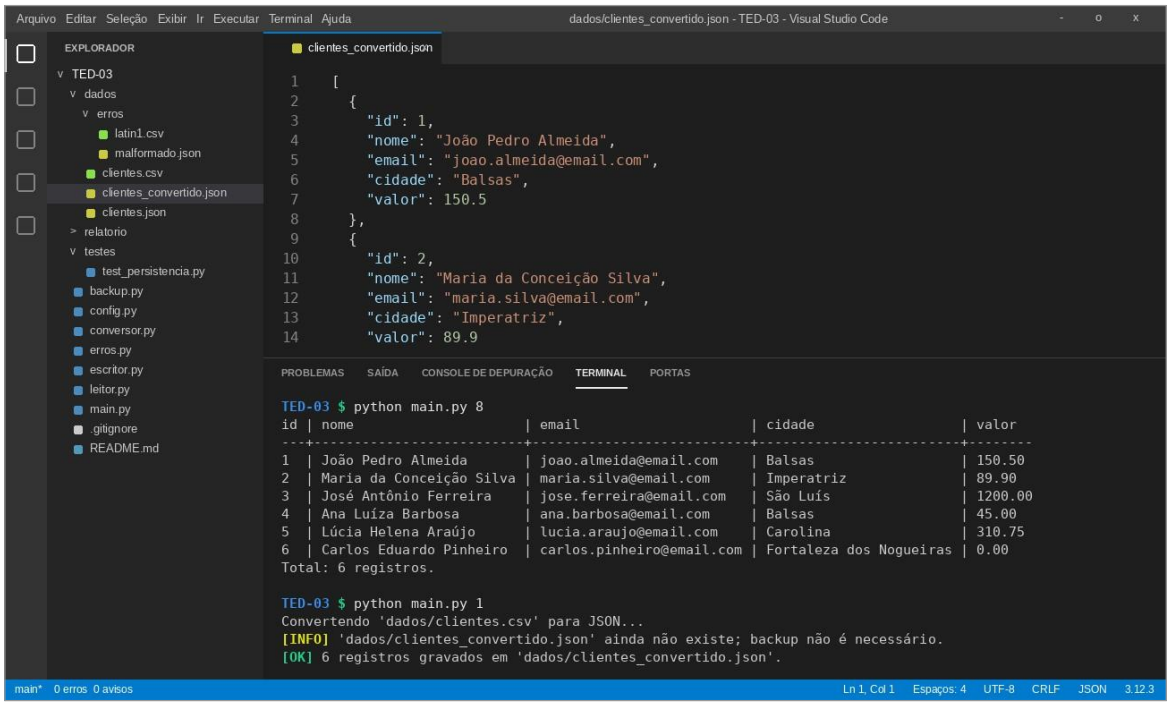
Executamos o programa no terminal integrado do Visual Studio Code, usando os arquivos de exemplo do repositório. Os testes manuais usam a opção correspondente do menu (aqui chamada diretamente, como em `python main.py 1`). Em cada cenário descrevemos o que foi feito e o resultado observado.

5.1 Leitura de um arquivo CSV e conversão para JSON

Contexto: o arquivo `dados/clientes.csv` contém 6 clientes, com nomes acentuados. Primeiro exibimos o arquivo com a opção 8, que usa a função de leitura, e depois executamos a opção 1 (CSV para JSON).

Resultado observado: o programa leu os 6 registros e gravou `dados/clientes_convertido.json`. Como o arquivo de saída ainda não existia, o programa informou que o backup não era necessário. No editor, o JSON gerado mostra `id` como inteiro, `valor` como número e os acentos preservados ("João Pedro Almeida", "Conceição").

Figura 2 – Leitura de clientes.csv e conversão para JSON (opções 8 e 1), com o arquivo gerado aberto no editor



Fonte: elaborada pelos autores (2026).

5.2 Leitura de um arquivo JSON e conversão para CSV

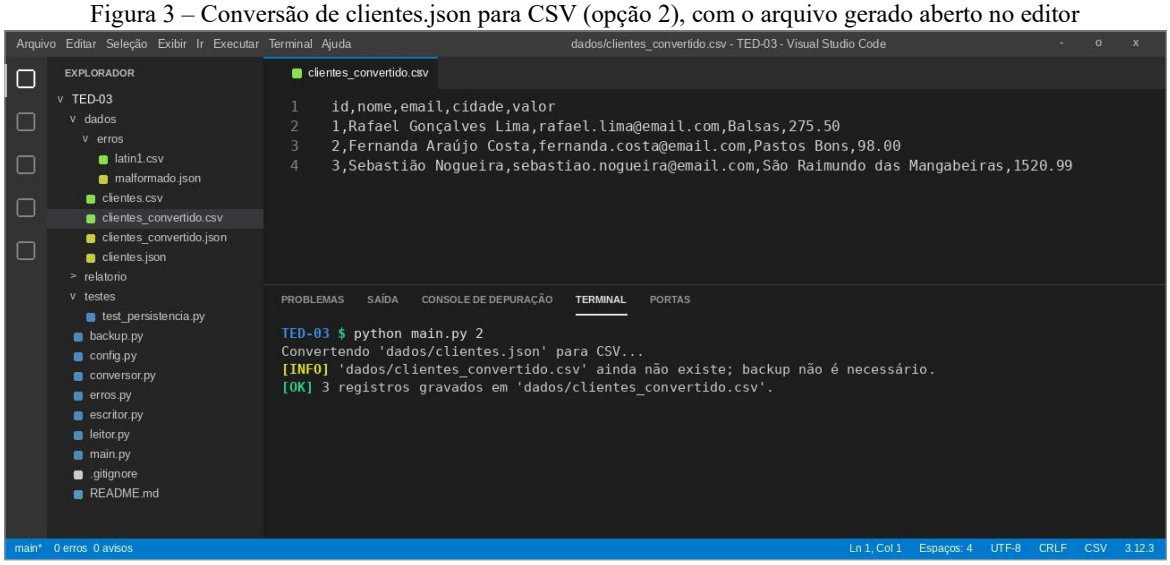
Contexto: o arquivo `dados/clientes.json` contém 3 clientes. Executamos a opção 2 (JSON para CSV).

Resultado observado: os 3 registros foram convertidos e gravados em `dados/clientes_convertido.csv`, com cabeçalho e acentos corretos ("Gonçalves",

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

"Araújo", "Sebastião"). Os valores decimais saíram com duas casas (275.50, 98.00, 1520.99), mantendo o formato monetário.

Figura 3 – Conversão de clientes.json para CSV (opção 2), com o arquivo gerado aberto no editor



Fonte: elaborada pelos autores (2026).

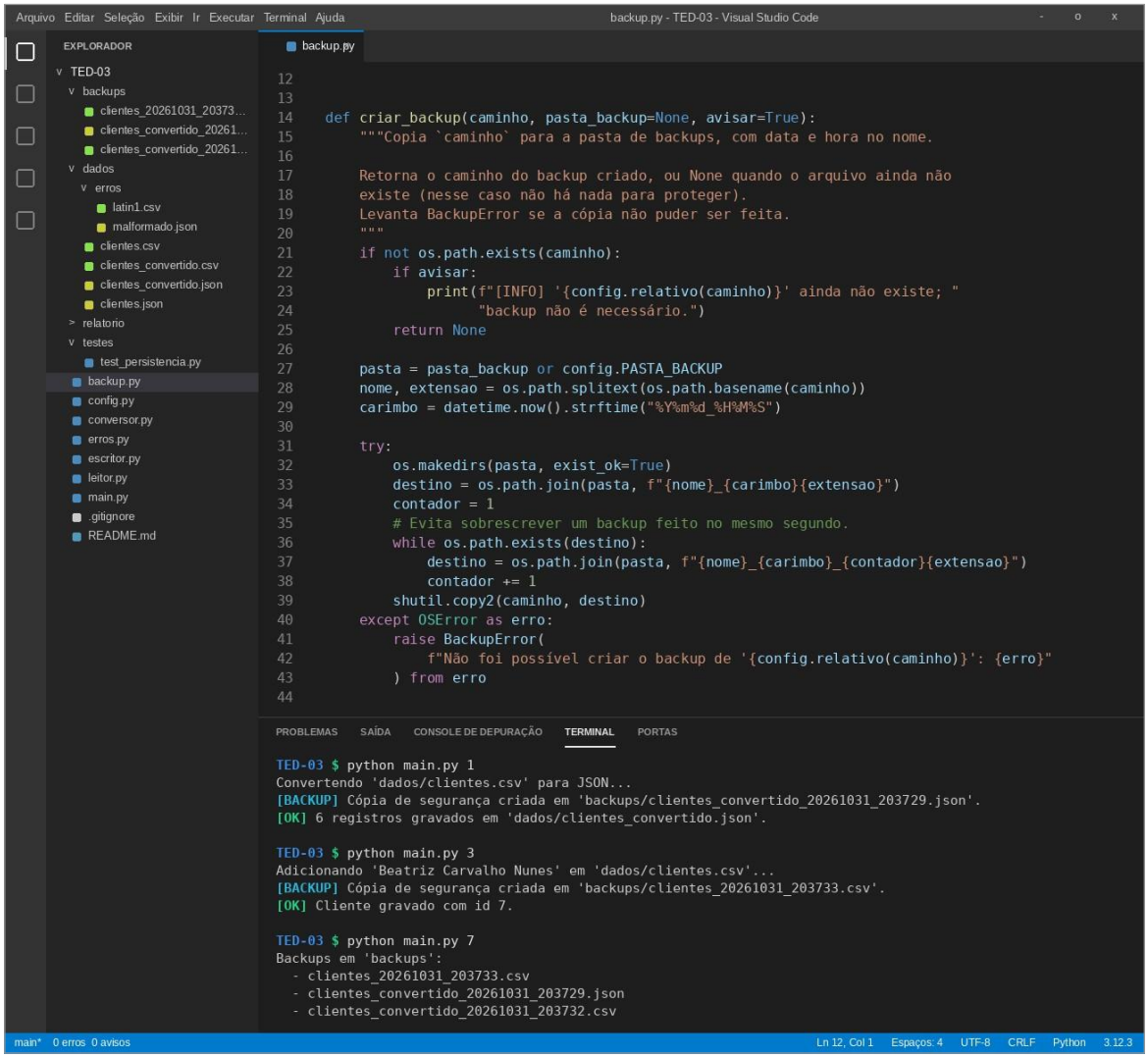
5.3 Escrita com geração prévia de backup

Contexto: para provar o backup, repetimos a opção 1. Agora `clientes_convertido.json` já existe e será sobrescrito. Em seguida usamos a opção 3, que adiciona um cliente em `dados/clientes.csv` (uma atualização do arquivo original), e a opção 7, que lista a pasta de backups.

Resultado observado: nas duas operações apareceu a mensagem [BACKUP] antes da confirmação [OK], indicando que a cópia foi feita antes da escrita. O novo cliente recebeu o id 7. A listagem mostra três arquivos em `backups/`, cada um com data e hora no nome: um do JSON convertido, um do CSV convertido (gerado quando repetimos a opção 2) e um do `clientes.csv` original.

Figura 4 – Backup automático antes da escrita (opções 1, 3 e 7), com o código de backup.py e a pasta backups/ no explorador

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br



Fonte: elaborada pelos autores (2026).

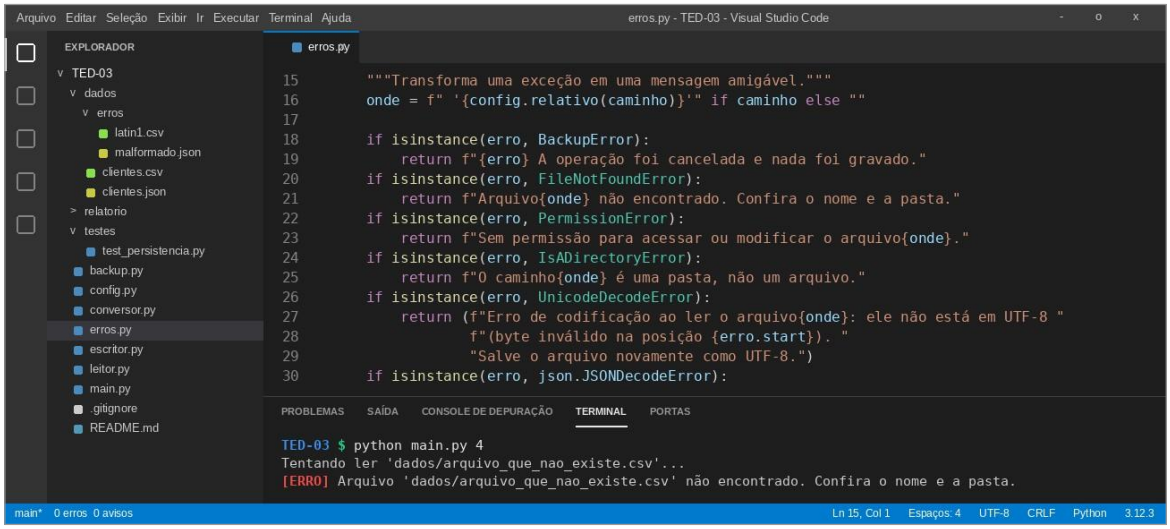
5.4 Tratamento de situações de erro de arquivo

Testamos três falhas diferentes. Em todas, o programa exibiu uma mensagem que identifica o problema e continuou funcionando, sem mostrar traceback.

a) Arquivo inexistente (opção 4). O programa tentou ler `dados/arquivo_que_nao_existe.csv`. Resultado: mensagem de arquivo não encontrado com a orientação de conferir o nome e a pasta.

Figura 5 – Tratamento de FileNotFoundError (opção 4), com a função mensagem_erro no editor

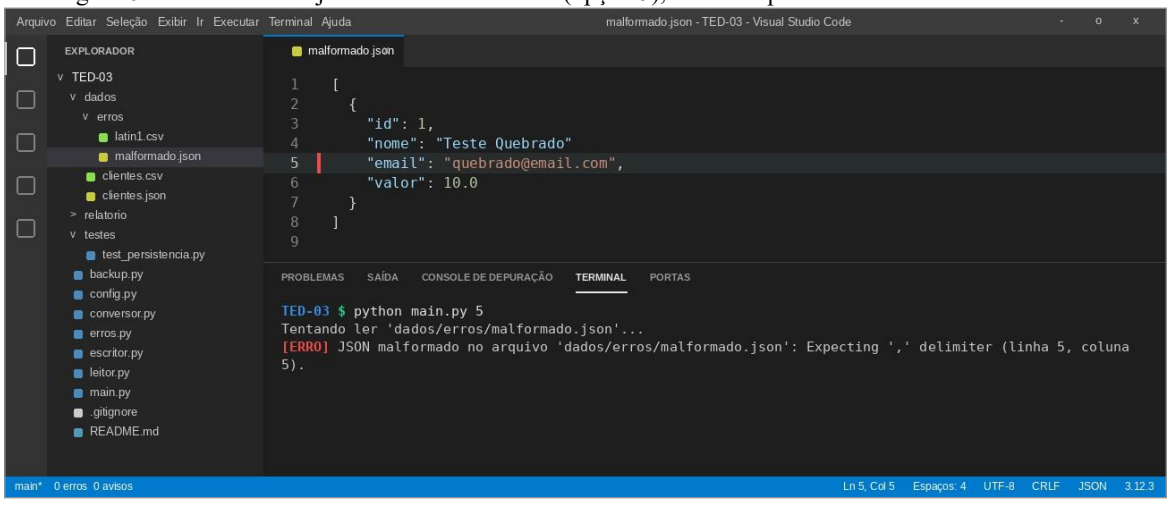
Km 5 da BR 230, Fazenda Maliderе IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br



Fonte: elaborada pelos autores (2026).

b) JSON malformado (opção 5). O arquivo `dados/erros/malformado.json` tem uma vírgula faltando depois do campo "nome". Resultado: o programa informou que o JSON está malformado e indicou a linha e a coluna onde o analisador encontrou o problema (linha 5, coluna 5, destacada no editor).

Figura 6 – Tratamento de json.JSONDecodeError (opção 5), com o arquivo malformado aberto no editor

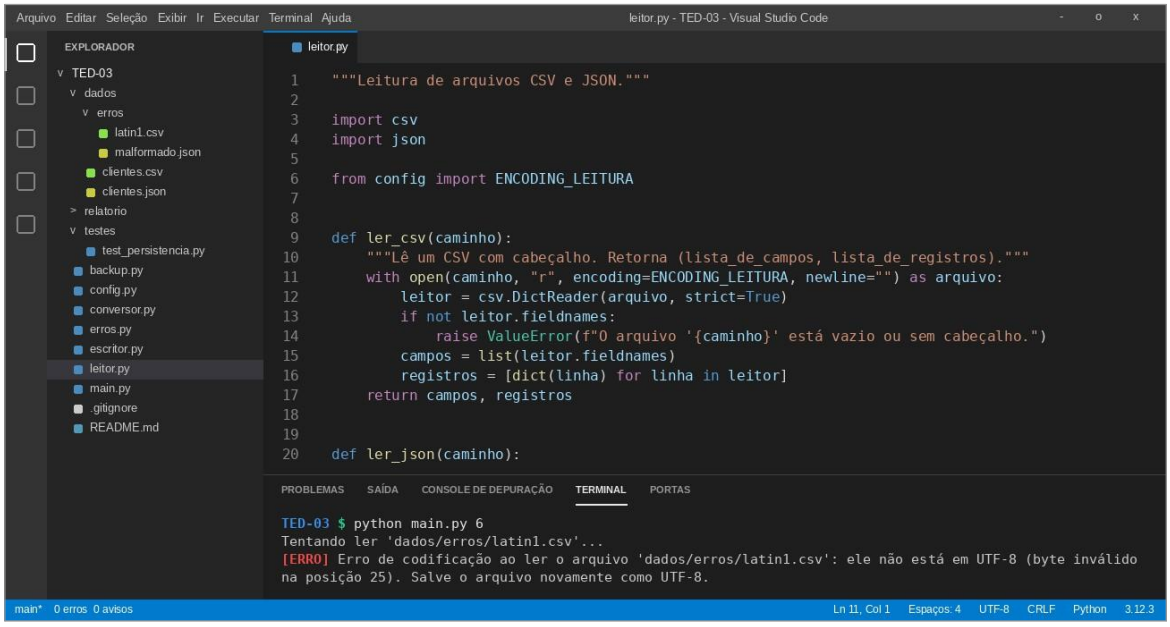


Fonte: elaborada pelos autores (2026).

c) Erro de codificação (opção 6). O arquivo `dados/erros/latin1.csv` foi salvo em Latin-1 e contém letras acentuadas. Como a leitura exige UTF-8, ocorre `UnicodeDecodeError`. Resultado: mensagem informando que o arquivo não está em UTF-8, com a posição do byte inválido e a sugestão de salvar novamente em UTF-8.

Figura 7 – Tratamento de UnicodeDecodeError (opção 6), com o leitor.py no editor

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br



Fonte: elaborada pelos autores (2026).

5.5 Testes automatizados

Além dos testes manuais, escrevemos 13 testes com `unittest`, executados com `python -m unittest discover -s testes -v`. Os testes usam pastas temporárias, por isso não alteram os arquivos do projeto. Para simular a falta de permissão, que é difícil de provocar da mesma forma em todos os sistemas, usamos `unittest.mock`.

Teste	O que verifica
test_csv_para_json_preserva_registros	Quantidade de registros, tipos (id inteiro, valor decimal) e nomes acentuados após CSV para JSON.
test_json_para_csv	Cabeçalho, campos e valores com duas casas decimais após JSON para CSV.
test_ida_e_volta_sem_perda	CSV para JSON e de volta para CSV gera um arquivo idêntico ao original.
test_acentos_preservados_no_arquivo_json	O JSON gravado contém “Conceição” e não a sequência de escape.
test_backup_criado_antes_de_sobrescrever	O backup existe e tem exatamente o conteúdo original.
test_arquivo_novo_nao_gera_backup	Arquivo que ainda não existe não cria backup nem pasta de backups.
test_falha_no_backup_impede_a_escrita	Se a cópia falhar, ocorre BackupError e o original continua intacto.

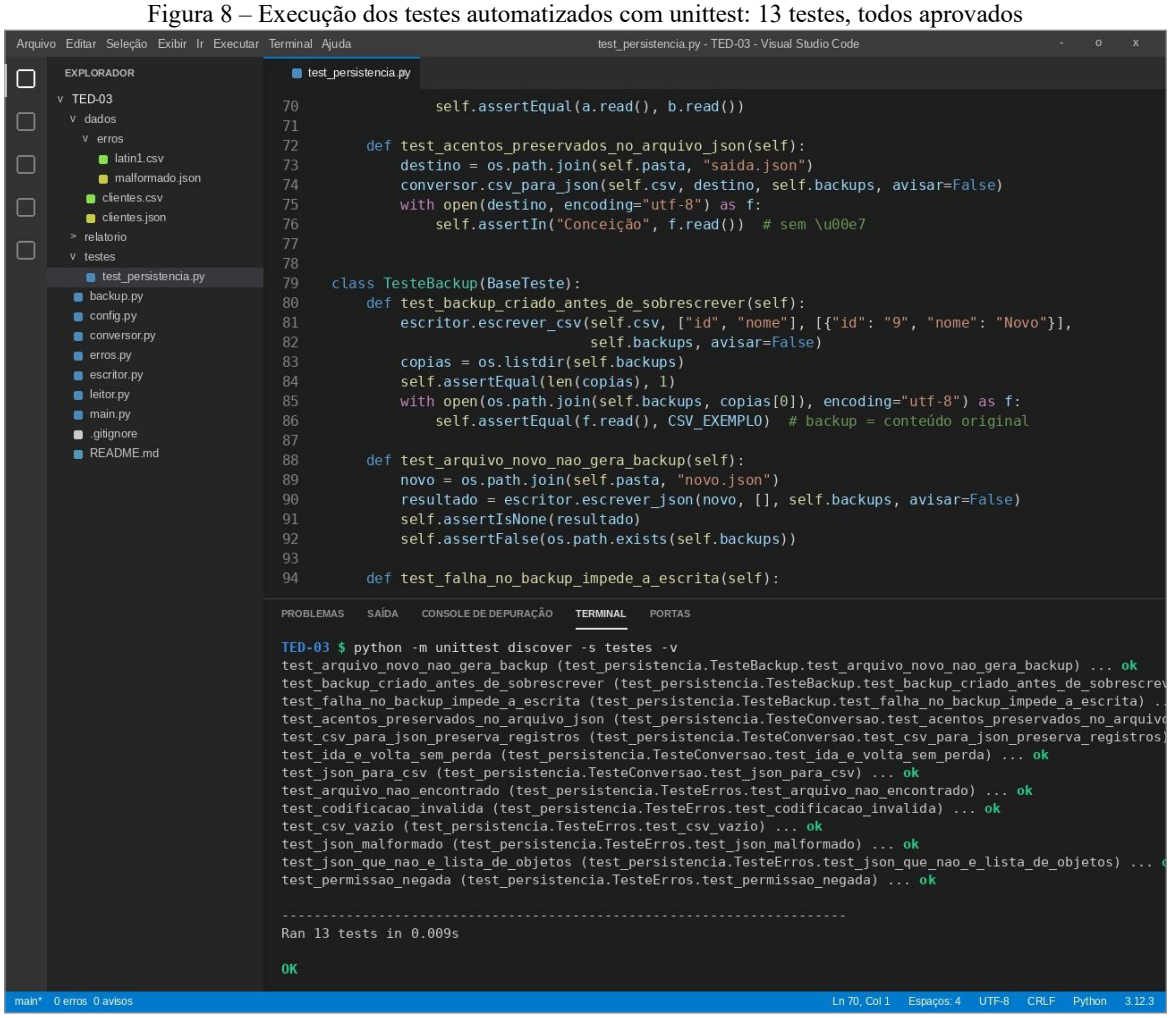
Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

test_arquivo_nao_encontrado	Mensagem de arquivo não encontrado.
test_permissao_negada	Mensagem de falta de permissão (simulada).
test_codificacao_invalida	Mensagem de erro de codificação para arquivo Latin-1.
Teste	**O que verifica**
test_json_malformado	Mensagem de JSON malformado.
test_json_que_nao_e_lista_de_objetos	Rejeita JSON que não seja uma lista de objetos.
test_csv_vazio	Rejeita CSV vazio ou sem cabeçalho.

Tabela 3 – Testes automatizados. Fonte: elaborada pelos autores (2026).

Resultado observado: os 13 testes foram executados e todos passaram (“Ran 13 tests ... OK”), como mostra a Figura 8.

Figura 8 – Execução dos testes automatizados com unittest: 13 testes, todos aprovados



Fonte: elaborada pelos autores (2026).

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

6. Conclusão

Os objetivos propostos foram alcançados. O módulo lê arquivos CSV e JSON, converte nos dois sentidos sem perder registros, grava em UTF-8 com os acentos preservados e usa `with open(...)` em todas as aberturas de arquivo. As falhas de arquivo mais comuns (arquivo inexistente, falta de permissão, codificação inválida, JSON malformado e CSV inválido) são tratadas com mensagens claras, e o programa não encerra de forma inesperada. O backup automático é criado antes de qualquer escrita sobre um arquivo existente, e a escrita é cancelada se o backup falhar.

Entre as dificuldades encontradas, a primeira foi preservar os valores na conversão. Como o CSV só guarda texto, converter "150.50" em número e voltar gerava "150.5", o que altera o formato de um valor monetário; resolvemos gravando os decimais com duas casas e confirmando com o teste de ida e volta. A segunda foi garantir que o backup acontecesse antes da alteração e que sua falha impedisse a gravação, o que nos levou a chamar o backup como primeira instrução das funções de escrita. A terceira foi testar situações que dependem do sistema, como a falta de permissão, o que resolvemos com simulação (`mock`). Também precisamos decidir como lidar com o BOM do Excel, resolvido com a codificação `utf-8-sig` na leitura.

Como possíveis melhorias futuras, o módulo poderia aceitar outros separadores de CSV, permitir que o usuário escolha os arquivos pelo menu, remover backups antigos de forma automática e tratar tipos de dados mais variados na conversão.

7. Referências

BRAY, T. (ed.). **The JavaScript Object Notation (JSON) Data Interchange Format**. RFC 8259. Internet Engineering Task Force, 2017. Disponível em: https://www.rfc-editor.org/rfc/rfc8259.

PYTHON SOFTWARE FOUNDATION. **csv — CSV File Reading and Writing**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/csv.html. Acesso em: out. 2026.

PYTHON SOFTWARE FOUNDATION. **json — JSON encoder and decoder**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/json.html. Acesso em: out. 2026.

PYTHON SOFTWARE FOUNDATION. **shutil — High-level file operations**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/shutil.html. Acesso em: out. 2026.

PYTHON SOFTWARE FOUNDATION. **Built-in Exceptions**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/exceptions.html. Acesso em: out. 2026.

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br

PYTHON SOFTWARE FOUNDATION. **unittest — Unit testing framework**. Python 3 Documentation. Disponível em: https://docs.python.org/3/library/unittest.html. Acesso em: out. 2026.

SHAFRANOVICH, Y. **Common Format and MIME Type for Comma-Separated Values (CSV) Files**. RFC 4180. Internet Engineering Task Force, 2005. Disponível em: https://www.rfc-editor.org/rfc/rfc4180.

Km 5 da BR 230, Fazenda Malidere IV
65800-000, Balsas-MA
99 3542-5500 - unibalsas. edu.br